

What Is the Second Pair of Eyes For?

One developer opens a pull request. Another reads it and approves it. We call that approval the second pair of eyes. Now the change may have been written by an AI, and since September 2026 an AI can count as the approver too.

Where did it come from?

Maintainer, reviewer, regulator.

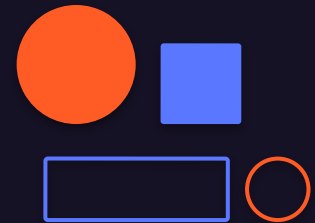
What changed?

Writing became cheap. Approval became a setting.

Who should own the check?

And what I would try first.

An introduction to how far I got. I am looking for reactions and disagreement, not a decision.



01

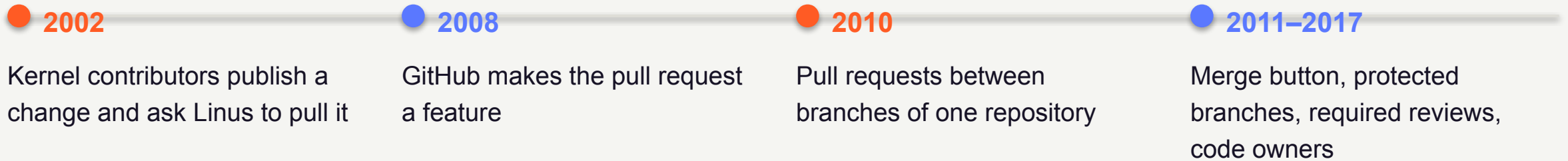
Where did the second pair of eyes come from?

Three people wanted a second reader, and each wanted something different from it. None of them designed the thing we use today.

- The pull request was a request to integrate
- Who lives with the change
- Defects, or understanding
- The regulator's pair of eyes
- One pull request does three jobs



The pull request was a request to integrate



Writing a change and accepting it were separate acts from the start. That was how the work was divided, not a control. The approval rules came later, on top.

Who lives with the change

The kernel

Linus read the code and carried the consequences himself.

A payment company

One developer reads the change. The company answers for it, to customers and to its supervisor.

A required approval puts a developer at the gate. It does not say what they should do there.

Why people wanted another person

82%

of errors caught by inspection in Fagan's 1976 case study

14%

of Microsoft review comments that pointed at a defect

29%

suggested improvements to code that already worked

Google's reason was different from the start: review exists so that other developers can **understand** the code. Two questions, one approval. Does it work, and will the next person understand it?

The regulator's pair of eyes

"The independence of the functions that approve changes and the functions responsible for requesting and implementing those changes."

Functions, not persons

A function is a role. Nothing says the approver must be a person.

Risk-based

A config change and a change that moves money need not get the same scrutiny.

Silent on AI

No supervisor has written that an AI may or may not approve a change.

One pull request does three jobs

The reading

The comments prove someone read it.

The gate

The approval is what the change has to pass.

The record

The pull request stays behind as evidence.

The weak point is the gate. The approver is usually a colleague on the same team. And nobody asked what happens when the changes outnumber the people who can read them.

02

What changed when writing became cheap?

More code arrives than anyone can read. The reader adapted, and the approval became a setting.

- Writing became cheap
- Familiarity replaced reading
- Three failures that look like one
- The numbers stopped adding up
- Four ways out
- The approval became a setting



Writing became cheap

+51%

changes per developer in
one year at Meta

25m → 90m

merged pull requests a
month on GitHub, 2023 to
2026

16 h vs 3 h

wait before an AI-written pull
request is picked up

400 vs 157

lines in an AI-written pull
request, decided in less time

Bigger changes, decided faster. Probably not read.

Familiarity replaced reading

30,1% →
36,8%

approval rate as reviewers
saw more agent-written code

−22%

inline comments over the
same seven months

96%

of developers do not fully
trust AI output

48%

always verify it before
committing

Reviewers approve what has become familiar. Authors submit what they doubt.

Three failures that look like one

1 · The reading is overwhelmed

More code arrives than people can read.

2 · The first reading never happened

The author skims 400 generated lines and opens the pull request. Nothing checks for that.

3 · The record lies

Approved, when nobody read it.

I approved a change because the tests were green, without reading it. The record shows my approval. What it recorded was confidence in the tests.

The numbers stopped adding up

200 lines / h

beyond this, reviewers find about half the defects

1 in 4

AI-written pull requests has more than 400 lines

2 hours

of one person's attention for that one change

3,2 h / week

spent reviewing by Google developers before AI

Nobody can review everything properly anymore. Most teams have already stopped reviewing some changes. **Did anyone choose which ones?**

Four ways out

Have fewer pull requests

Godot banned AI-generated code and kept the human approver.

Approve some without a human

Zalando auto-approves 33% as low-risk. None of these companies is regulated.

Check before you open it

Coding agents review your branch first. Rust: not a substitute for self-review.

Make each one simpler to approve

Adyen: under five files, a human approves, a designated person re-reads.

The approval became a setting

Copilot, the author

Blocked from approving, like any author.

Copilot, the reviewer

A second identity. It can approve what the first one wrote, and the audit log has no event for it.

Someone has to decide which repositories have it on, which changes are exempt, and who answers when it goes wrong. On most teams nobody has that job.

03

Who should own the check?

One shared reviewer for everyone is the obvious answer. It is better at two jobs out of three.

- One answer: a centralized AI reviewer
- What it would cost
- Centralize the machinery, keep the decision local



One answer: a centralized AI reviewer

"An outsider sees more"

The research measures context, not reading.

"AI never gets tired"

The people reading its reviews do.

"Nobody looks for security"

Because nobody asked them to.

Better at coverage. Possibly better at the record. Not better at the approval.

What it would cost

**\$43.000 to
\$260.000**

a year in licences for about
300 engineers

>90%

of the riskiest changes
approved by the FCA's 23
firms' change boards

2,6×

more likely to be a worst
performer with outside
approvers

32 of 33

planted vulnerabilities
passed two AI reviewers

A service that reads every change stops being a check. And every repository misses the same bugs.

Centralize the machinery, keep the decision local

Mechanically verifiable

Formatting, known bug patterns, secrets, dependencies.
Google applies 3.000 automated fixes a day at under 5% false positives.

Needs a person

Is the change wanted? Norms, gatekeeping, education. Correct code can still be unwanted.

Atlassian replaced 20 CI/CD servers with one platform and cut lead time from 2 days to 90 minutes. Cloudflare runs one reviewer with per-team config and a logged break-glass. **Shared tools, local approval.**

04

What would I try first?

Five things, cheapest first. Four of them exist today. The fifth is mine and unproven.

- What I would try before building a review service
- The author's half of four eyes



What I would try before building a review service

1. **Turn on what already exists.** Required reviews, status checks, secret and dependency scanning. I have not checked what is on today.
2. **Tier repositories by what they touch.** More from the ones that move money, less from the internal chatbot.
3. **Centralize the machinery.** Formatters, linters, scanners, once for the company.
4. **Pair models across vendors.** A different model family reviews what the first one wrote.
5. **The author's half of four eyes.** The one new idea. Next slide.

The author's half of four eyes

Before opening a pull request, the author explains the change in their own words.

Why

Explaining something yourself measurably deepens your understanding.

Limit

Across 80.000 pull requests, descriptions rarely changed the review. Except when they explained and asked.

Risk

A required field decays into "N/A".

Status: Prototype 0.1 is a design. No team has used it.

05

What would I do in a repository?

Chapter 04 was the order for the company. This is the same order in one repository, with the file each step lives in.

- First, decide what this repository touches
- Make the mechanical checks unskippable
- Two decisions that follow from the tier
- Ask the author for what no check can see
- Where each step lives



First, decide what this repository touches

Chapter 04 tiered repositories and never said who does the tiering. This repository has to answer for itself, because the answer sets how strict everything after it should be.

- Does it move money, hold customer data, or neither?
- Write the answer in the README, under a short "Review rules" heading
- Say what follows: how many reviewers, whether a machine may approve, whether a security review is mandatory
- Revisit it when the repository starts touching something new

One sentence in the README that a new joiner can find is more than most repositories have today.

Make the mechanical checks unskippable

Run the checks

Format, lint, test and static analysis in a workflow file. Dependency and secret scanning switched on.

Require the checks

Require each check by name. Require a pull request. Stop direct pushes to the default branch.

Keep approvals current

Dismiss stale approvals when new commits land, so the approval refers to the code that actually merges.

A check that runs is a suggestion. A check that blocks a merge is a control.

Two decisions that follow from the tier

Who reviews the paths that carry risk

A CODEOWNERS file requires a named reviewer for the code that moves money or touches customer data. The rest of the repository stays on the lighter default.

Whether a machine may approve here

Off by default today. A payment repository and an internal dashboard may answer differently. Neither should answer by accident.

Write it in the README next to the tier, with the reason. The setting has nowhere to hold one, and nothing logs a machine approval.

Ask the author for what no check can see

One pull request template. The same questions in front of every change.

- What changed, and why it belongs here
- What you want looked at, specifically
- How to undo it
- Which parts an agent wrote

Across 80.000 pull requests, descriptions rarely changed the review. The exception was one that explains the code and asks for a specific kind of feedback. So ask for both.

Where each step lives

Step	Do	Where it lives
01	Write down what this repository touches, and what follows from it	README, under "Review rules"
02	Format, lint, test, static analysis, dependency and secret scanning	.github/workflows/, .github/dependabot.yml, settings
03	Require those checks, require a pull request, stop direct pushes, dismiss stale approvals	Branch protection, or a ruleset
04	A named reviewer for the paths that carry risk	.github/CODEOWNERS
05	Whether a machine may approve here, and why	README, under "Review rules"
06	The template that explains and asks	.github/pull_request_template.md
07	Read ten merged pull requests and ask what you are seeing	A calendar entry, not a tool

Every step but the last is a file or a setting, copied to the next repository in minutes and reverted by deleting a line. The last one is half an hour a month. **I have not checked which of these are already on in ours.**



Conclusion

What moves, what doesn't, and what I still don't know.



What moves, and what doesn't

Shorter queue

Turn on what is there. Centralize the mechanical. Log it. Tier by risk.

Higher quality, with evidence

A different model family reviews the change. Two more are design problems, not results.

Set aside

Restricting AI use. Open source defends against strangers. We employ our authors.

The approval stays with a human who is independent of whoever implemented the change. That is the one line that does not move.

What I don't know yet

Same team, or outside it?

Everything here assumes same-team approval is enough. The regulation does not say.

Does a classifier count as a function?

Meta and Zalando auto-approve. Nobody has written down whether DORA allows it.

What is switched on today?

A day of checking I have not done.

Does the author's check survive a team?

Nobody has used it.

What would help me most

A review decision you made and had to undo. What did it cost?

What would change my mind

A regulated company whose auditor accepted a machine's approval.

A1 — Who did what about the queue

Use when someone asks for the full list behind slide 13.

Who	What they did	Human approver?	Regulated?
Meta	Risk score per change, low-risk merged automatically. 331.000+ to production.	No, for low risk	No
Zalando	Classifier on open. 33% auto-approved as low-risk, author merges.	No, for low risk	No
Spotify	76% more pull requests. Auto-merges what it judges safe. Soundcheck tiers checks per component type.	No, for safe	No
Shopify	System writes and explains security fixes, freshness-gated merge queue. Backlog down ~70% in 11 days.	Yes	No
Adyen	4.000+ automated merge requests from a deterministic bot, under five files each. Designated person re-reads approved changes.	Yes	Yes
Monzo	"An engineer still reviews and merges, but the bottleneck moves from 'find an engineer with capacity' to 'find a reviewer'."	Yes	Yes
Cloudflare	131.000+ review runs in 30 days across 5.000+ repositories. Per-team config, break-glass used 0,6% of the time. Explicitly not a replacement for human review.	Yes	No
Godot	Banned AI-generated contributions. "All PRs must be reviewed and approved by a human before merging."	Yes	No
Rust (five teams)	Circuit breaker: AI over half of merges in six weeks, AI changes stop for ten days.	Yes	No
curl	Ended its bug bounty in January 2026. About 20% of security submissions were slop.	Yes	No
GitHub	Per-contributor limit on open pull requests. Copilot approval can count toward required approvals since 1 Sep 2026.	Setting	No

A2 — The regulatory detail

Use when someone wants the rule read out rather than summarised.

- **Two persons direct the business.** European banking law (CRD art. 13) grants a licence only where "at least two persons effectively direct the business". That is the origin of Vier-Augen-Prinzip and the English phrase four eyes. They approve no individual change.
- **Dual control on a sensitive action.** Basel defines it as two or more separate "entities (usually persons)" acting in concert. The Norwegian supervisor fined a savings bank in 2023 partly because no four-eyes check existed when an employee manually lowered a customer's money-laundering risk level.
- **Change management before July 2025.** IKT-forskriften § 9 required procedures for handling changes, and that they be followed. It never asked for an approver.
- **Change management since July 2025.** DORA (EU 2022/2554) art. 9(4)(e), with RTS 2024/1774 art. 17(1)(b): "the independence of the functions that approve changes and the functions responsible for requesting and implementing those changes". Every change recorded, tested, assessed, approved, implemented and verified. Policy "based on a risk assessment approach". Source code reviews defined as static and dynamic testing.
- **Not defined:** function, independence. Elsewhere in the rulebook a function is a part of the organisation. Whether the approver must sit outside the implementing team is not stated.

The art. 17 wording relies on two secondary reproductions that agree. Verify against EUR-Lex before anything is quoted externally. Two things in this deck are called DORA: the EU act here, and Google's DevOps Research and Assessment on slides 17 and 18.

A3 — How far to trust each number

Use when someone pushes on a figure. Every number in the main line has a caveat, and these are them.

Number	Where it comes from	How far it carries
30,1% → 36,8% approval, comments –22%	400 reviewers, 11.429 reviews, seven months, three controls. KDD 2026 workshop paper.	Measures repetition and load, not team boundaries.
96% distrust, 48% always verify	Sonar survey, about 1.150 respondents.	Self-reported.
200 lines an hour	Kemerer & Paulk, 2009.	One study. The 2-hour arithmetic is mine and rests on it.
3,2 h and 6,4 h a week reviewing	Google telemetry; open-source survey of 287 respondents.	The open-source figure is self-reported.
62% vs 53,7%; 60% vs 50,5%	Greptile, two sets of 500 pull requests, July 2026.	Vendor research selling the feature. Directional only.
32 of 33 vulnerabilities passed review	Preprint, March 2026, 33 CVEs across 20 projects.	Not peer-reviewed. The mechanism carries even if the number moves.
Fewer unreviewed PRs, fewer security bugs	3.126 projects, nearly 400.000 pull requests, PROMISE 2017.	Association, open source only. Effect called small but significant.
2,6× more likely worst performers	Google's DORA (DevOps Research and Assessment), 2019.	Correlation across survey respondents.
>90% of riskiest changes approved	UK FCA multi-firm review, 23 firms, over a million changes, 2021.	Major changes are selected into board review for being risky.
Close time 6 h → 8 h after turning a reviewer on	One company, 4.335 pull requests, ICSE 2025 industry track.	One company.
Atlassian 75% / 96% / \$4 million	Atlassian engineering blog, August 2025.	First-party. A 2026 review finds no proof platforms cause such gains.
60 to 70% of platform initiatives fail	Conference reporting in The New Stack, October 2025.	Industry reporting, not a study. About platforms in general, not review.

A4 — Names and terms

Use if the room mixes people who live in the tooling with people who don't.

Term	Meaning in this deck
Four eyes	Three different rules: two persons run a bank, a second person checks a sensitive action, the approving function is independent of the implementing one.
DORA (EU)	Digital Operational Resilience Act, EU 2022/2554. Applies to payment institutions since July 2025.
DORA (Google)	DevOps Research and Assessment. Google Cloud's research programme on software delivery. Coined "the verification tax" in 2026.
RTS	Regulatory Technical Standard. The detailed rules under DORA (EU). 2024/1774 art. 17 is the change-management one.
Required approvals, code owners	GitHub branch-protection rules from 2016 and 2017. One more person has to say yes before a merge.
Break-glass	A logged override that lets a team bypass a check in an emergency. Cloudflare's reviewer has one; GitHub's org-wide rulesets do not.
Tricorder	Google's static-analysis platform. Runs on every change before a human reviewer sees it.
Soundcheck	Spotify's platform for running different tiers of checks per kind of component.
Self-review	The author reading their own change as a critic would before asking anyone else to. GitHub recommends it. Rust's policy says an LLM review is not a substitute.
The author's half of four eyes	My name for a check that asks the author to explain the change in their own words before opening a pull request. A design, not yet a result.